

JAVA BASICS

Complete Notes: Loops & Arrays

Includes clear explanations, Java code examples, sample outputs,
and 20 practice questions with answers.

Estimated study time: 2+ hours

Prepared by: Akshit Sharma

Phone: 8510968871 | Email: akshubhardwaj231@gmail.com

Website: akshbuilds.tech

Table of Contents

- 1. Introduction to Loops
- 2. The for Loop
- 3. The while Loop
- 4. The do-while Loop
- 5. The Enhanced for-each Loop
- 6. Nested Loops
- 7. break and continue Statements
- 8. Introduction to Arrays
- 9. Declaring and Initializing 1D Arrays
- 10. Traversing / Accessing Arrays
- 11. Common Array Operations (sum, max, min, reverse, search)
- 12. Two-Dimensional (2D) Arrays
- 13. Arrays + Loops together (real practice)
- 14. 20 Practice Questions with Answers

SECTION 1: LOOPS IN JAVA

1. Introduction to Loops

A **loop** is used in Java when you want to repeat a block of code multiple times without writing it again and again. Instead of writing `System.out.println()` 10 times to print 10 numbers, a loop can do it in just 3-4 lines.

Java provides four main ways to repeat code: **for loop**, **while loop**, **do-while loop**, and the **enhanced for-each loop**. Every loop has three important parts:

- **Initialization** - starting point of the loop (e.g. `i = 1`)
- **Condition** - the test that is checked before every repetition (e.g. `i <= 10`)
- **Update/Increment** - how the counter changes after each repetition (e.g. `i++`)

As long as the condition is **true**, the loop keeps repeating (this is called an *iteration*). The moment the condition becomes **false**, the loop stops.

2. The for Loop

The **for loop** is used when you already know exactly how many times you want to repeat something. All three parts (initialization, condition, update) are written together in one line, which makes the code short and clean.

Syntax

```
for (initialization; condition; update) {  
    // code to repeat (loop body)  
}
```

How it works step by step: initialization runs once → condition is checked → if true, the body runs → update runs → condition is checked again → this repeats until condition is false.

Example: Print numbers from 1 to 10

Java Code Example

```
public class ForLoopExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            System.out.println("Number: " + i);  
        }  
    }  
}
```

Output

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
Number: 6
Number: 7
Number: 8
Number: 9
Number: 10
```

Example: Sum of first N natural numbers

Java Code Example

```
public class SumExample {
    public static void main(String[] args) {
        int n = 5, sum = 0;
        for (int i = 1; i <= n; i++) {
            sum = sum + i;
        }
        System.out.println("Sum = " + sum);
    }
}
```

Output

```
Sum = 15
```

Note: A for loop can also count downwards, e.g. `for(int i = 10; i >= 1; i--)`, or jump in steps, e.g. `i = i + 2` to print only even numbers.

3. The while Loop

The **while loop** is used when you do NOT know exactly how many times the loop should run in advance - it keeps running as long as the condition stays true. The condition is checked **before** every iteration, so if the condition is false at the very start, the loop body never runs even once.

Syntax

```
initialization;
while (condition) {
    // code to repeat
    update;
}
```

Example: Print numbers from 1 to 5

Java Code Example

```
public class WhileLoopExample {
    public static void main(String[] args) {
        int i = 1;
        while (i <= 5) {
            System.out.println("i = " + i);
            i++;
        }
    }
}
```

Output

```
i = 1
i = 2
i = 3
i = 4
i = 5
```

Common use case: *while loops are great when the number of repetitions depends on user input or some changing condition, like reading data until a sentinel value (e.g. -1) is entered.*

4. The do-while Loop

The **do-while loop** is similar to the while loop, but with one key difference: the condition is checked **after** the loop body runs. This guarantees the loop body executes **at least once**, even if the condition is false from the beginning.

Syntax

```
do {
    // code to repeat
} while (condition);    // note the semicolon at the end
```

Example

Java Code Example

```
public class DowhileExample {
    public static void main(String[] args) {
        int i = 1;
        do {
            System.out.println("Value: " + i);
            i++;
        } while (i <= 5);
    }
}
```

Output

```
Value: 1
Value: 2
Value: 3
Value: 4
Value: 5
```

Real-life example: *Menu-driven programs use do-while so the menu is shown at least once before checking whether the user wants to exit.*

5. The Enhanced for-each Loop

The **for-each loop** (also called enhanced for loop) is used to go through every element of an array or collection one by one, without needing an index variable. It is shorter and less error-prone than a normal for loop when you only need to read values (not their index).

Syntax

```
for (dataType variable : arrayOrCollection) {  
    // use variable  
}
```

Example

Java Code Example

```
public class ForEachExample {  
    public static void main(String[] args) {  
        int[] marks = {90, 85, 76, 60, 99};  
        for (int m : marks) {  
            System.out.println("Mark: " + m);  
        }  
    }  
}
```

Output

```
Mark: 90  
Mark: 85  
Mark: 76  
Mark: 60  
Mark: 99
```

6. Nested Loops

A **nested loop** is a loop written inside another loop. The inner loop completes ALL of its iterations for every single iteration of the outer loop. Nested loops are commonly used for patterns, tables, and working with 2D arrays.

Example: Print a multiplication-style pattern

Java Code Example

```
public class NestedLoopExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                System.out.print(i * j + "\t");  
            }  
            System.out.println();  
        }  
    }  
}
```

Output

```
1 2 3  
2 4 6  
3 6 9
```

Example: Star pattern

Java Code Example

```
public class StarPattern {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print("*");  
            }  
            System.out.println();  
        }  
    }  
}
```

Output

```
*  
**  
***  
****  
*****
```

7. break and continue Statements

break is used to completely stop (exit) a loop as soon as it is encountered - control jumps to the line right after the loop.

Java Code Example

```
public class BreakExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i == 5) {  
                break; // loop stops completely when i == 5  
            }  
            System.out.println(i);  
        }  
    }  
}
```

Output

```
1  
2  
3  
4
```

continue is used to skip only the current iteration and move on to the next one - the loop does NOT stop.

Java Code Example

```
public class ContinueExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 6; i++) {  
            if (i % 2 == 0) {  
                continue;    // skip even numbers  
            }  
            System.out.println(i);  
        }  
    }  
}
```

Output

```
1  
3  
5
```

SECTION 2: ARRAYS IN JAVA

8. Introduction to Arrays

An **array** is a container object that holds a **fixed number of values of the same data type** in a single variable, instead of creating separate variables for each value. Each value in an array is called an **element**, and each element is accessed using a number called its **index**.

- Array indexing in Java always starts from **0**, not 1.
- If an array has 5 elements, valid indexes are 0, 1, 2, 3, 4.
- The size of an array is fixed once it is created - it cannot grow or shrink.
- All elements of an array must be of the **same data type** (e.g. all int, or all String).

9. Declaring and Initializing 1D Arrays

Method 1: Declare, then create with a fixed size

Java Code Example

```
int[] numbers = new int[5];    // creates an array of 5 integers, all set to 0
numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;
```

Method 2: Declare and initialize with values directly

Java Code Example

```
int[] numbers = {10, 20, 30, 40, 50};
String[] fruits = {"Apple", "Banana", "Mango"};
double[] prices = {99.5, 45.0, 120.75};
```

Default values: If you create an array with new and don't assign values, Java fills it with defaults automatically: **0** for int/double, **false** for boolean, and **null** for String/objects.

Example: Finding the length of an array

Java Code Example

```
public class ArrayLengthExample {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        System.out.println("Array length: " + numbers.length);
        System.out.println("First element: " + numbers[0]);
        System.out.println("Last element: " + numbers[numbers.length - 1]);
    }
}
```

Output

```
Array length: 5
First element: 10
Last element: 50
```

10. Traversing / Accessing Arrays

"Traversing" an array means visiting every element one by one, which is usually done using a for loop (when the index is needed) or a for-each loop (when only the value is needed).

Java Code Example

```
public class TraverseArray {  
    public static void main(String[] args) {  
        int[] marks = {88, 76, 90, 65, 59};  
  
        // Using a normal for loop (index available)  
        for (int i = 0; i < marks.length; i++) {  
            System.out.println("Index " + i + " -> " + marks[i]);  
        }  
    }  
}
```

Output

```
Index 0 -> 88  
Index 1 -> 76  
Index 2 -> 90  
Index 3 -> 65  
Index 4 -> 59
```

Important: Trying to access an index that does not exist (e.g. marks[10] on a 5-element array) causes an `ArrayIndexOutOfBoundsException` - one of the most common beginner errors in Java.

11. Common Array Operations

a) Sum and Average of array elements

Java Code Example

```
public class SumAverage {  
    public static void main(String[] args) {  
        int[] arr = {12, 45, 67, 23, 89};  
        int sum = 0;  
        for (int i = 0; i < arr.length; i++) {  
            sum += arr[i];  
        }  
        double average = (double) sum / arr.length;  
        System.out.println("Sum = " + sum);  
        System.out.println("Average = " + average);  
    }  
}
```

Output

```
Sum = 236  
Average = 47.2
```

b) Finding the Maximum and Minimum element

Java Code Example

```
public class MaxMinExample {
    public static void main(String[] args) {
        int[] arr = {12, 45, 67, 23, 89, 5};
        int max = arr[0], min = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) max = arr[i];
            if (arr[i] < min) min = arr[i];
        }
        System.out.println("Maximum = " + max);
        System.out.println("Minimum = " + min);
    }
}
```

Output

```
Maximum = 89
Minimum = 5
```

c) Reversing an array

Java Code Example

```
public class ReverseArray {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        System.out.print("Reversed: ");
        for (int i = arr.length - 1; i >= 0; i--) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

Output

```
Reversed: 5 4 3 2 1
```

d) Linear Search (finding an element)

Java Code Example

```
public class LinearSearch {
    public static void main(String[] args) {
        int[] arr = {10, 25, 30, 45, 60};
        int key = 30;
        boolean found = false;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key) {
                System.out.println("Found at index: " + i);
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Element not found");
        }
    }
}
```

Output

```
Found at index: 2
```

12. Two-Dimensional (2D) Arrays

A **2D array** is basically an "array of arrays" - it stores data in rows and columns, like a table or a grid. It is useful for representing matrices, seating layouts, game boards, etc.

Declaration and initialization:

Java Code Example

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
// matrix[row][column]
// matrix[0][0] = 1, matrix[1][2] = 6, matrix[2][1] = 8
```

Traversing a 2D array using nested loops:

Java Code Example

```
public class TwoDArrayExample {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        for (int row = 0; row < matrix.length; row++) {
            for (int col = 0; col < matrix[row].length; col++) {
                System.out.print(matrix[row][col] + " ");
            }
            System.out.println();
        }
    }
}
```

Output

```
1 2 3
4 5 6
7 8 9
```

13. Arrays + Loops Together (Real Practice)

Loops and arrays almost always go hand-in-hand in Java. Below is a combined example that fills an array using a loop, and then processes it - this is the pattern you will use constantly in real programs.

Java Code Example

```
import java.util.Scanner;

public class CombinedExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];

        // Fill array using a loop
        for (int i = 0; i < arr.length; i++) {
            arr[i] = i * 10;    // simulated input: 0,10,20,30,40
        }

        // Process array using another loop
        int sum = 0;
        for (int val : arr) {
            sum += val;
        }
        System.out.println("Total = " + sum);
    }
}
```

Output

```
Total = 100
```

SECTION 3: 20 CODING QUESTIONS

Each question below has a short explanation of the logic, the complete runnable Java code (with class and main method), and a step-by-step dry run showing exactly how the variables change during execution. Try coding it yourself first, then compare. Questions 1-10 cover loops, questions 11-20 cover arrays.

Q1. Print all numbers from 1 to N using a loop (N = 5)

Explanation: Use a for loop starting at 1, running while the counter is less than or equal to N, printing the counter and increasing it by 1 each time.

Full Java Code

```
public class PrintNumbers {  
    public static void main(String[] args) {  
        int n = 5;  
        for (int i = 1; i <= n; i++) {  
            System.out.print(i + " ");  
        }  
    }  
}
```

Dry Run (step-by-step trace)

```
n = 5  
i=1 -> 1<=5 true -> print 1 -> i becomes 2  
i=2 -> 2<=5 true -> print 2 -> i becomes 3  
i=3 -> 3<=5 true -> print 3 -> i becomes 4  
i=4 -> 4<=5 true -> print 4 -> i becomes 5  
i=5 -> 5<=5 true -> print 5 -> i becomes 6  
i=6 -> 6<=5 false -> loop stops  
Final Output: 1 2 3 4 5
```

Q2. Print all even numbers between 1 and N (N = 10)

Explanation: Loop from 1 to N and use the modulus operator (%) to check if a number is divisible by 2 with no remainder; if so, it is even.

Full Java Code

```
public class EvenNumbers {  
    public static void main(String[] args) {  
        int n = 10;  
        for (int i = 1; i <= n; i++) {  
            if (i % 2 == 0) {  
                System.out.print(i + " ");  
            }  
        }  
    }  
}
```

Dry Run (step-by-step trace)

```
i=1 -> 1%2=1 -> not even -> skip
i=2 -> 2%2=0 -> even -> print 2
i=3 -> 3%2=1 -> skip
i=4 -> 4%2=0 -> even -> print 4
i=5 -> skip ... i=6 -> print 6 ... i=7 -> skip
i=8 -> print 8 ... i=9 -> skip ... i=10 -> print 10
Final Output: 2 4 6 8 10
```

Q3. Find the sum of the first N natural numbers (N = 5)

Explanation: Start a variable sum at 0, then loop from 1 to N, adding the counter to sum on every iteration.

Full Java Code

```
public class SumOfN {
    public static void main(String[] args) {
        int n = 5, sum = 0;
        for (int i = 1; i <= n; i++) {
            sum = sum + i;
        }
        System.out.println("Sum = " + sum);
    }
}
```

Dry Run (step-by-step trace)

```
sum=0
i=1 -> sum = 0+1 = 1
i=2 -> sum = 1+2 = 3
i=3 -> sum = 3+3 = 6
i=4 -> sum = 6+4 = 10
i=5 -> sum = 10+5 = 15
i=6 -> 6<=5 false -> loop stops
Final Output: Sum = 15
```

Q4. Find the factorial of a number (N = 5)

Explanation: Factorial of N (N!) is the product of all whole numbers from 1 to N. Start a variable fact at 1, then multiply it by the loop counter on every iteration.

Full Java Code

```
public class Factorial {
    public static void main(String[] args) {
        int n = 5;
        long fact = 1;
        for (int i = 1; i <= n; i++) {
            fact = fact * i;
        }
        System.out.println("Factorial = " + fact);
    }
}
```

Dry Run (step-by-step trace)

```
fact=1
i=1 -> fact = 1*1 = 1
i=2 -> fact = 1*2 = 2
i=3 -> fact = 2*3 = 6
i=4 -> fact = 6*4 = 24
i=5 -> fact = 24*5 = 120
Final Output: Factorial = 120
```

Q5. Check whether a number is prime (N = 29)

Explanation: A prime number has no divisors other than 1 and itself. Loop from 2 up to the square root of N; if any number divides N exactly, it is not prime.

Full Java Code

```
public class PrimeCheck {
    public static void main(String[] args) {
        int n = 29;
        boolean isPrime = true;
        if (n < 2) {
            isPrime = false;
        }
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                isPrime = false;
                break;
            }
        }
        System.out.println(n + " is Prime: " + isPrime);
    }
}
```

Dry Run (step-by-step trace)

```
n = 29, isPrime = true
i=2 -> 2*2=4 <= 29 -> 29%2=1 (not 0) -> continue
i=3 -> 3*3=9 <= 29 -> 29%3=2 (not 0) -> continue
i=4 -> 4*4=16 <= 29 -> 29%4=1 (not 0) -> continue
i=5 -> 5*5=25 <= 29 -> 29%5=4 (not 0) -> continue
i=6 -> 6*6=36 > 29 -> loop stops
No divisor found, isPrime stays true
Final Output: 29 is Prime: true
```

Q6. Print the Fibonacci series up to N terms (N = 8)

Explanation: Fibonacci series: each number is the sum of the previous two (0, 1, 1, 2, 3, 5...). Keep two variables for the previous two terms and update them every iteration.

Full Java Code

```

public class Fibonacci {
    public static void main(String[] args) {
        int n = 8, a = 0, b = 1;
        for (int i = 1; i <= n; i++) {
            System.out.print(a + " ");
            int next = a + b;
            a = b;
            b = next;
        }
    }
}

```

Dry Run (step-by-step trace)

```

a=0, b=1
i=1 -> print 0 -> next=0+1=1 -> a=1, b=1
i=2 -> print 1 -> next=1+1=2 -> a=1, b=2
i=3 -> print 1 -> next=1+2=3 -> a=2, b=3
i=4 -> print 2 -> next=2+3=5 -> a=3, b=5
i=5 -> print 3 -> next=3+5=8 -> a=5, b=8
i=6 -> print 5 ... i=7 -> print 8 ... i=8 -> print 13
Final Output: 0 1 1 2 3 5 8 13

```

Q7. Reverse a given number (N = 1234)

Explanation: Repeatedly take the last digit of the number using % 10, add it to a reversed number (building it from the left), then remove the last digit from N using / 10.

Full Java Code

```

public class ReverseNumber {
    public static void main(String[] args) {
        int n = 1234, reversed = 0;
        while (n != 0) {
            int digit = n % 10;
            reversed = reversed * 10 + digit;
            n = n / 10;
        }
        System.out.println("Reversed = " + reversed);
    }
}

```

Dry Run (step-by-step trace)

```

n=1234, reversed=0
digit=1234%10=4 -> reversed=0*10+4=4 -> n=1234/10=123
digit=123%10=3 -> reversed=4*10+3=43 -> n=123/10=12
digit=12%10=2 -> reversed=43*10+2=432 -> n=12/10=1
digit=1%10=1 -> reversed=432*10+1=4321 -> n=1/10=0
n=0 -> loop stops
Final Output: Reversed = 4321

```

Q8. Check whether a number is a palindrome (N = 121)

Explanation: A palindrome reads the same forwards and backwards. Reverse the number (same logic as above) and compare the reversed value with the original number.

Full Java Code

```
public class PalindromeCheck {
    public static void main(String[] args) {
        int n = 121, original = n, reversed = 0;
        while (n != 0) {
            int digit = n % 10;
            reversed = reversed * 10 + digit;
            n = n / 10;
        }
        if (original == reversed) {
            System.out.println(original + " is a Palindrome");
        } else {
            System.out.println(original + " is NOT a Palindrome");
        }
    }
}
```

Dry Run (step-by-step trace)

```
original=121, n=121, reversed=0
digit=121%10=1 -> reversed=0*10+1=1 -> n=121/10=12
digit=12%10=2 -> reversed=1*10+2=12 -> n=12/10=1
digit=1%10=1 -> reversed=12*10+1=121 -> n=1/10=0
n=0 -> loop stops
Compare: original(121) == reversed(121) -> true
Final Output: 121 is a Palindrome
```

Q9. Print the multiplication table of a number (N = 7)

Explanation: Loop the multiplier from 1 to 10 and print N multiplied by the current multiplier on each line.

Full Java Code

```
public class MultiplicationTable {
    public static void main(String[] args) {
        int n = 7;
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + " x " + i + " = " + (n * i));
        }
    }
}
```

Dry Run (step-by-step trace)

```
i=1 -> 7 x 1 = 7
i=2 -> 7 x 2 = 14
i=3 -> 7 x 3 = 21
...
i=10 -> 7 x 10 = 70
(loop runs exactly 10 times, i from 1 to 10)
```

Q10. Find the sum of digits of a number (N = 4567)

Explanation: Repeatedly extract the last digit using % 10, add it to a running sum, then remove the last digit using / 10, until the number becomes 0.

Full Java Code

```
public class SumOfDigits {  
    public static void main(String[] args) {  
        int n = 4567, sum = 0;  
        while (n != 0) {  
            int digit = n % 10;  
            sum = sum + digit;  
            n = n / 10;  
        }  
        System.out.println("Sum of digits = " + sum);  
    }  
}
```

Dry Run (step-by-step trace)

```
n=4567, sum=0  
digit=4567%10=7 -> sum=0+7=7 -> n=4567/10=456  
digit=456%10=6 -> sum=7+6=13 -> n=456/10=45  
digit=45%10=5 -> sum=13+5=18 -> n=45/10=4  
digit=4%10=4 -> sum=18+4=22 -> n=4/10=0  
n=0 -> loop stops  
Final Output: Sum of digits = 22
```

Q11. Find the sum and average of array elements ({12, 45, 67, 23, 89})

Explanation: Loop through every element of the array, adding each one to a sum variable, then divide the sum by the array length to get the average.

Full Java Code

```
public class SumAverageArray {  
    public static void main(String[] args) {  
        int[] arr = {12, 45, 67, 23, 89};  
        int sum = 0;  
        for (int i = 0; i < arr.length; i++) {  
            sum += arr[i];  
        }  
        double average = (double) sum / arr.length;  
        System.out.println("Sum = " + sum);  
        System.out.println("Average = " + average);  
    }  
}
```

Dry Run (step-by-step trace)

```
sum=0  
i=0 -> arr[0]=12 -> sum=0+12=12  
i=1 -> arr[1]=45 -> sum=12+45=57  
i=2 -> arr[2]=67 -> sum=57+67=124  
i=3 -> arr[3]=23 -> sum=124+23=147  
i=4 -> arr[4]=89 -> sum=147+89=236  
i=5 -> 5<5 false -> loop stops  
average = 236 / 5 = 47.2  
Final Output: Sum = 236, Average = 47.2
```

Q12. Find the largest and smallest element in an array ({4, 19, 2, 77, 33})

Explanation: Assume the first element is both the max and min, then compare every other element against them, updating whenever a bigger or smaller value is found.

Full Java Code

```
public class MaxMinArray {
    public static void main(String[] args) {
        int[] arr = {4, 19, 2, 77, 33};
        int max = arr[0], min = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) max = arr[i];
            if (arr[i] < min) min = arr[i];
        }
        System.out.println("Max = " + max);
        System.out.println("Min = " + min);
    }
}
```

Dry Run (step-by-step trace)

```
max=4, min=4 (arr[0])
i=1 -> arr[1]=19 -> 19>4 -> max=19 | 19<4? no
i=2 -> arr[2]=2 -> 2>19? no | 2<4 -> min=2
i=3 -> arr[3]=77 -> 77>19 -> max=77 | 77<2? no
i=4 -> arr[4]=33 -> 33>77? no | 33<2? no
Final Output: Max = 77, Min = 2
```

Q13. Search for an element using Linear Search ({10, 25, 30, 45, 60}, key = 45)

Explanation: Check every element one by one from the start of the array; if it matches the key, print its index and stop searching.

Full Java Code

```
public class LinearSearch {
    public static void main(String[] args) {
        int[] arr = {10, 25, 30, 45, 60};
        int key = 45;
        boolean found = false;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key) {
                System.out.println("Found at index: " + i);
                found = true;
                break;
            }
        }
        if (!found) {
            System.out.println("Element not found");
        }
    }
}
```

Dry Run (step-by-step trace)

```
key = 45
i=0 -> arr[0]=10 -> 10==45? no
i=1 -> arr[1]=25 -> 25==45? no
i=2 -> arr[2]=30 -> 30==45? no
i=3 -> arr[3]=45 -> 45==45? yes -> print index, found=true, break
Final Output: Found at index: 3
```

Q14. Reverse an array ({1, 2, 3, 4, 5})

Explanation: Start a loop from the last index and move backward to index 0, printing each element as it is visited.

Full Java Code

```
public class ReverseArray {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        System.out.print("Reversed: ");
        for (int i = arr.length - 1; i >= 0; i--) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

Dry Run (step-by-step trace)

```
arr.length=5, start i=4
i=4 -> arr[4]=5 -> print 5
i=3 -> arr[3]=4 -> print 4
i=2 -> arr[2]=3 -> print 3
i=1 -> arr[1]=2 -> print 2
i=0 -> arr[0]=1 -> print 1
i=-1 -> -1>=0 false -> loop stops
Final Output: Reversed: 5 4 3 2 1
```

Q15. Count even and odd numbers in an array ({3, 8, 11, 20, 7, 16})

Explanation: Loop through the array and use % 2 on every element; increase an evenCount or oddCount variable depending on the result.

Full Java Code

```

public class CountEvenOdd {
    public static void main(String[] args) {
        int[] arr = {3, 8, 11, 20, 7, 16};
        int evenCount = 0, oddCount = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] % 2 == 0) {
                evenCount++;
            } else {
                oddCount++;
            }
        }
        System.out.println("Even count = " + evenCount);
        System.out.println("Odd count = " + oddCount);
    }
}

```

Dry Run (step-by-step trace)

```

evenCount=0, oddCount=0
3  -> odd  -> oddCount=1
8  -> even -> evenCount=1
11 -> odd  -> oddCount=2
20 -> even -> evenCount=2
7   -> odd  -> oddCount=3
16 -> even -> evenCount=3
Final Output: Even count = 3, Odd count = 3

```

Q16. Find the second largest element in an array ({12, 35, 1, 10, 34, 1})

Explanation: Keep two variables, largest and secondLargest. Update both while scanning the array: if a value beats largest, push largest down into secondLargest first.

Full Java Code

```

public class SecondLargest {
    public static void main(String[] args) {
        int[] arr = {12, 35, 1, 10, 34, 1};
        int largest = Integer.MIN_VALUE, secondLargest = Integer.MIN_VALUE;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] > largest) {
                secondLargest = largest;
                largest = arr[i];
            } else if (arr[i] > secondLargest && arr[i] != largest) {
                secondLargest = arr[i];
            }
        }
        System.out.println("Largest = " + largest);
        System.out.println("Second Largest = " + secondLargest);
    }
}

```

Dry Run (step-by-step trace)

```

largest=MIN, secondLargest=MIN
12 -> 12>MIN -> secondLargest=MIN, largest=12
35 -> 35>12 -> secondLargest=12, largest=35
1  -> 1>35? no -> 1>12? no -> skip
10 -> 10>35? no -> 10>12? no -> skip
34 -> 34>35? no -> 34>12? yes -> secondLargest=34
1  -> skip
Final Output: Largest = 35, Second Largest = 34

```

Q17. Sort an array in ascending order using Bubble Sort ({5, 2, 9, 1, 5})

Explanation: Repeatedly compare each pair of adjacent elements and swap them if they are in the wrong order. After each full pass, the largest remaining element 'bubbles' to the end.

Full Java Code

```

public class BubbleSort {
    public static void main(String[] args) {
        int[] arr = {5, 2, 9, 1, 5};
        for (int i = 0; i < arr.length - 1; i++) {
            for (int j = 0; j < arr.length - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
        for (int val : arr) {
            System.out.print(val + " ");
        }
    }
}

```

Dry Run (step-by-step trace)

```

Start: [5, 2, 9, 1, 5]
Pass 1: 5>2 swap ->[2,5,9,1,5] | 5>9? no | 9>1 swap ->[2,5,1,9,5] | 9>5 swap ->[2,5,1,5,9]
Pass 2: 2>5? no | 5>1 swap ->[2,1,5,5,9] | 5>5? no
Pass 3: 2>1 swap ->[1,2,5,5,9]
Pass 4: 1>2? no
Final Output: 1 2 5 5 9

```

Q18. Find the frequency of a given element in an array ({4, 2, 4, 7, 4, 9, 2}, key = 4)

Explanation: Loop through the array and increase a counter every time the current element equals the key value.

Full Java Code

```

public class FrequencyCount {
    public static void main(String[] args) {
        int[] arr = {4, 2, 4, 7, 4, 9, 2};
        int key = 4, count = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key) {
                count++;
            }
        }
        System.out.println("Frequency of " + key + " = " + count);
    }
}

```

Dry Run (step-by-step trace)

```

key=4, count=0
4 -> match -> count=1
2 -> no
4 -> match -> count=2
7 -> no
4 -> match -> count=3
9 -> no
2 -> no
Final Output: Frequency of 4 = 3

```

Q19. Add two matrices using 2D arrays (2x2 matrices)

Explanation: Use a nested loop to go through every row and column; add the corresponding elements of both matrices and store the result in a third matrix.

Full Java Code

```

public class MatrixAddition {
    public static void main(String[] args) {
        int[][] a = {{1, 2}, {3, 4}};
        int[][] b = {{5, 6}, {7, 8}};
        int[][] sum = new int[2][2];

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                sum[i][j] = a[i][j] + b[i][j];
            }
        }
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                System.out.print(sum[i][j] + " ");
            }
            System.out.println();
        }
    }
}

```

Dry Run (step-by-step trace)

```
i=0,j=0 -> sum[0][0]=a[0][0]+b[0][0]=1+5=6
i=0,j=1 -> sum[0][1]=a[0][1]+b[0][1]=2+6=8
i=1,j=0 -> sum[1][0]=a[1][0]+b[1][0]=3+7=10
i=1,j=1 -> sum[1][1]=a[1][1]+b[1][1]=4+8=12
Final Output:
6 8
10 12
```

Q20. Find the sum of all elements in a 2D array ({1,2,3},{4,5,6},{7,8,9})

Explanation: Use a nested loop (outer for rows, inner for columns) to visit every element of the 2D array exactly once, adding each one to a running total.

Full Java Code

```
public class Sum2DArray {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        int sum = 0;
        for (int row = 0; row < matrix.length; row++) {
            for (int col = 0; col < matrix[row].length; col++) {
                sum += matrix[row][col];
            }
        }
        System.out.println("Total sum = " + sum);
    }
}
```

Dry Run (step-by-step trace)

```
sum=0
row=0 -> 1,2,3 -> sum=0+1+2+3=6
row=1 -> 4,5,6 -> sum=6+4+5+6=21
row=2 -> 7,8,9 -> sum=21+7+8+9=45
Final Output: Total sum = 45
```

Prepared by: Akshit Sharma

Phone: 8510968871 | Email: akshubhardwaj231@gmail.com

Website: akshbuilds.tech